IMMO SCOPE

Analyse du marché immobilier — DVF 2023

Partie 3

Visualisation, Dashboard & Analyse Exploratoire

Hadjer Benaissa

Développement Logiciel — Data Visualisation

Projet HAX712X — Université Montpellier

2025/2026

1 Mon rôle dans le projet Immo Scope

Ma contribution au projet Immo Scope constitue la troisième et dernière partie du pipeline : la **visualisation des données** et la **construction du tableau de bord interactif**. Après le travail de nettoyage, d'ingénierie des données et de validation réalisé dans les parties précédentes, mon rôle a été d'utiliser les données fiables pour produire :

- des visualisations explicatives permettant de lire rapidement les tendances du marché ;
- une carte interactive basée sur la géolocalisation réelle des transactions ;
- une interface utilisateur complète développée avec Streamlit ;
- des outils visuels destinés à faciliter la prise de décision en immobilier.

Création des modules Python

J'ai développé deux modules majeurs :

- `Visualizer.py` : génération des graphiques statistiques (camembert, histogramme, top communes, relation surface–prix).
- `MapVisualizer.py` : création d'une carte Plotly Mapbox incluant une choroplèthe et un scatter géolocalisé.

Ces deux classes ont été conçues de manière autonome mais compatibles avec l'application Streamlit, en respectant une structure POO déjà établie dans le projet.

Développement du Dashboard Streamlit

L'application Streamlit représente la partie visible du projet. J'ai conçu :

- une page d'accueil présentant les résultats clés et la répartition des rôles ;
- une page de visualisation regroupant les 4 graphiques principaux ;
- une carte interactive filtrable par département ;
- une synthèse complète intégrant plusieurs vues simultanées.

Le travail a également consisté à améliorer l'expérience utilisateur :

- organisation en onglets pour une navigation fluide ;
- mise en cache via `@st.cache_resource` pour optimiser les performances ;
- choix de couleurs harmonisées et d'une interface cohérente ;
- ajout de messages d'erreur et de contrôles pour éviter les plantages.

Objectif global de ma partie

L'enjeu principal était de **rendre les données compréhensibles**, même pour un public non spécialisé. Mes visualisations permettent :

- d'interpréter le marché en quelques secondes ;
- d'explorer les transactions à plusieurs niveaux (global, local, géographique) ;
- d'obtenir un outil interactif directement utilisable.

2 Difficultés rencontrées et solutions apportées

1. Données hétérogènes et difficiles à représenter

Les données DVF présentent une forte variabilité : écarts de prix, surfaces incohérentes, communes peu renseignées. J'ai dû implémenter des filtres robustes côté visualisation pour éviter les graphiques trompeurs.

Solution : Échantillonnage intelligent, suppression des valeurs aberrantes dans les graphiques, regroupement des communes.

2. Besoin d'une interface fluide et ergonomique

Il fallait permettre à l'utilisateur d'explorer des centaines de transactions sans complexité.

Solution : Création d'une application **Streamlit multi-pages** avec menus, filtres, onglets et une navigation intuitive.

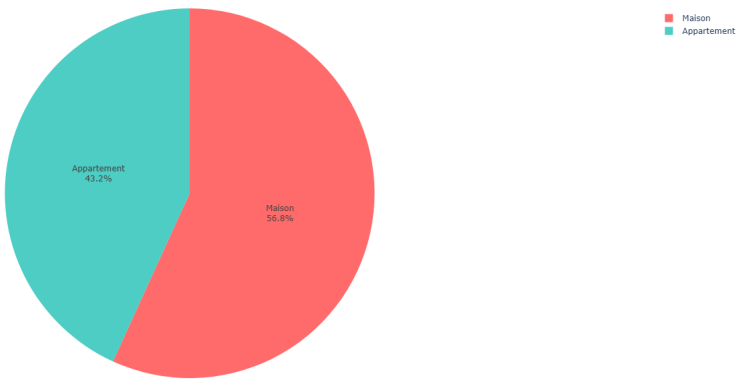
3. Intégration Mapbox et géocoordonnées

La carte interactive nécessite une cohérence parfaite entre communes, coordonnées et styles Mapbox.

Solution : Création d'une classe `MapVisualizer` dédiée + géométrie GeoJSON.

3 Visualisations produites

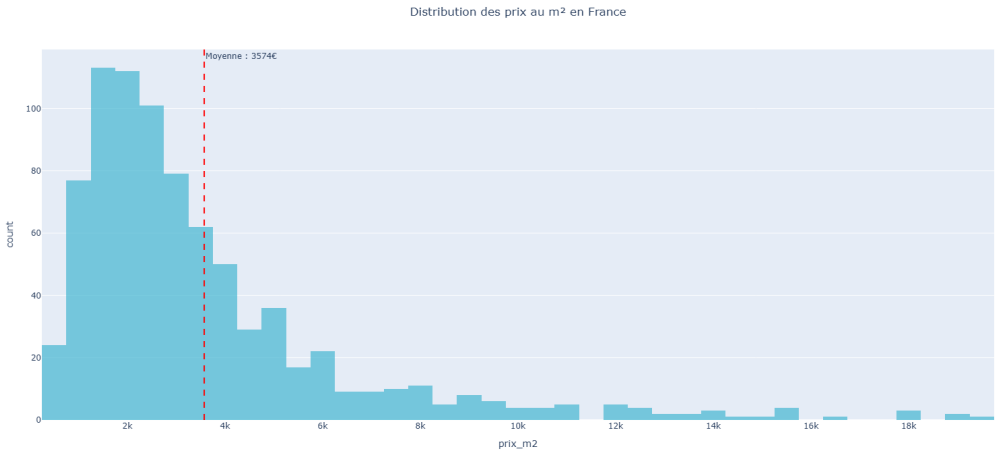
3.1 Répartition des types de biens



Rôle : Comprendre la composition du marché.

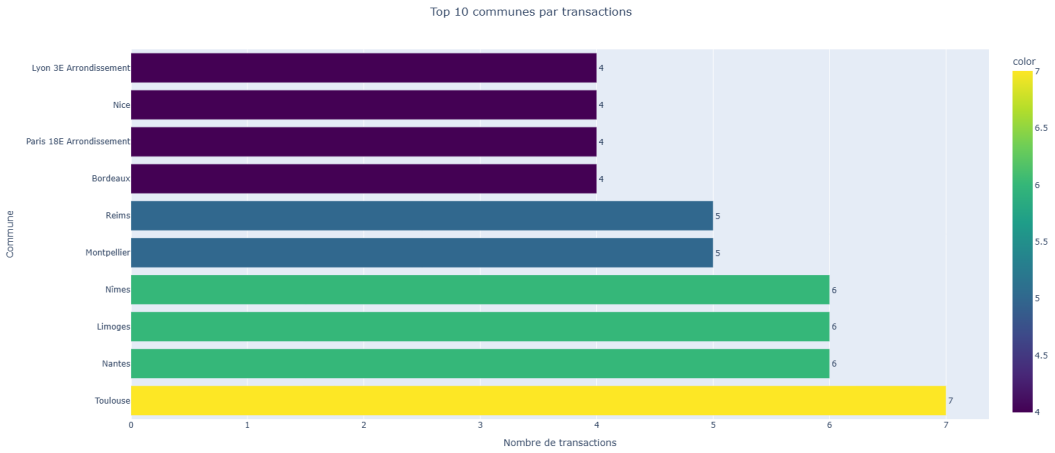
Observation : Les appartements dominent légèrement les ventes.

3.2 Distribution des prix au m²



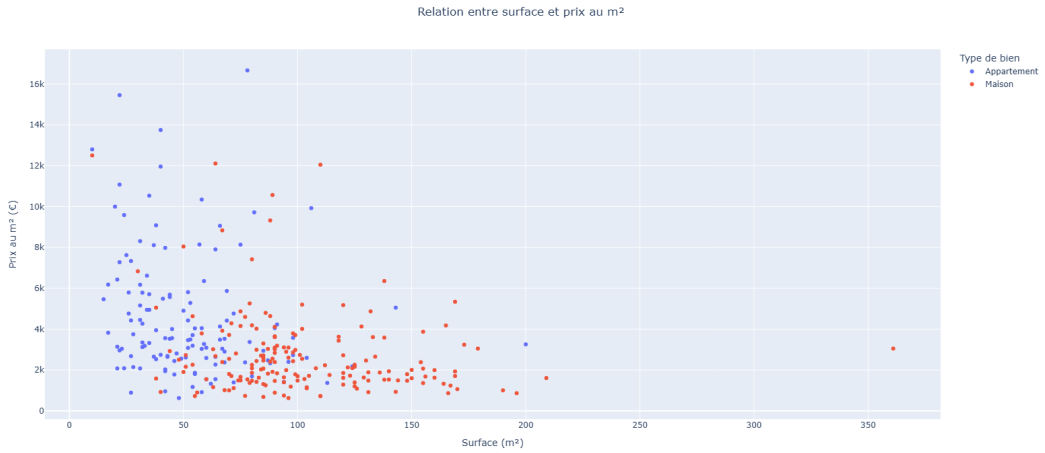
Rôle : Analyser la dispersion des prix.
Observation : La moyenne nationale se situe à environ 3574 €/m².

3.3 Top communes



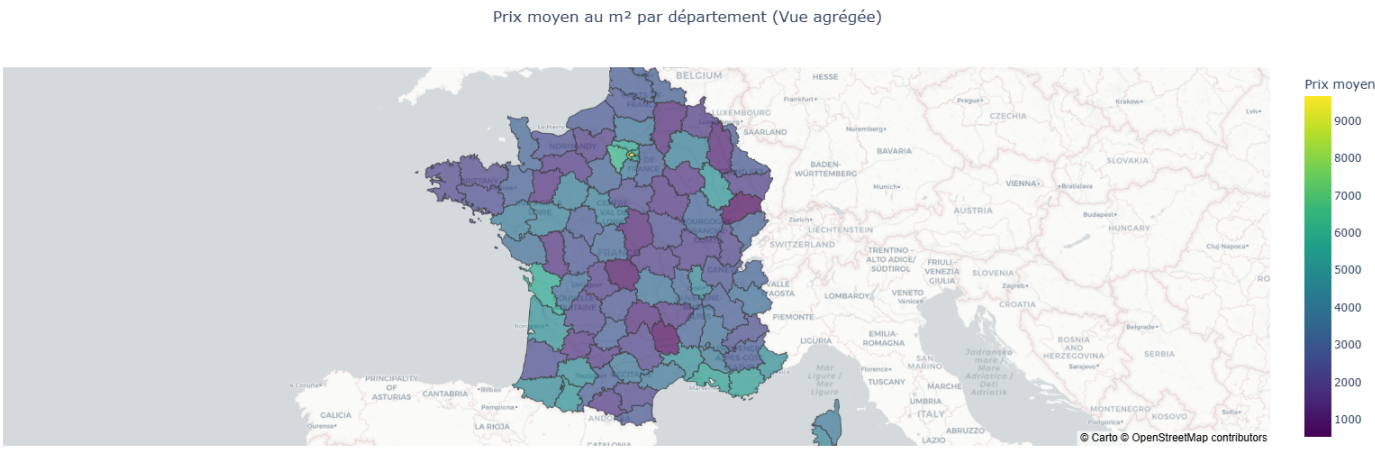
Rôle : Identifier les zones les plus actives.
Observation : Forte concentration de ventes dans quelques communes.

3.4 Surface vs Prix



Rôle : Détecter relations et anomalies.
Observation : Les petites surfaces affichent les prix/m² les plus élevés.

4 Carte Interactive



Cette carte offre :
— visualisation géographique réelle des transactions ;
— filtres dynamiques (département, commune) ;
— affichage interactif au survol ;
— représentation exploitable pour l'analyse spatiale.
Elle constitue la partie la plus visuelle et intuitive de l'application.

5 Code source développé

5.1 Module Visualizer.py

```
"""
MODULE␣DE␣VISUALISATION
D veloppement␣du␣dashboard␣interactif
"""
import json
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from pathlib import Path

class Visualizer:
    def __init__(self):
        """Initialise␣le␣visualizer␣avec␣les␣donn es"""
        current_dir = Path(__file__).parent
        project_root = current_dir.parent

        dashboard_file = project_root / "data" / "reports" / "
            dashboard_data.json"
        data_file = project_root / "data" / "processed" / "dvf_cleaned.csv
            "

        print("Visualizer␣initialis ␣avec␣succ s!")

        #Les donn es agr g es
        with open(dashboard_file, 'r', encoding='utf-8') as f:
            self.data = json.load(f)
        print("Donn es␣dashboard␣charg es")

        #Les donn es compl tes
        self.df = pd.read_csv(data_file)
        print(f"{len(self.df)}␣transactions␣charg es")

        # Afficher un r sum
        print(f"R sum :␣{self.data['metrics']['total_transactions']}␣
            transactions")
        print(f"Prix␣moyen:␣{self.data['metrics']['avg_price_m2']}␣  /m
            ")

    # ------------------------------------------------------------------
    #  Graphique : R partition des types de biens
    # ------------------------------------------------------------------
    def create_property_types_chart(self):

        if "type_local" not in self.df.columns:
            print("␣Colonne␣'type_local'␣manquante")
            return None

        stats = self.df["type_local"].value_counts()
```

```
        fig = px.pie(
            values=stats.values,
            names=stats.index,
            title="␣R partition␣des␣types␣de␣biens␣immobiliers",
            color_discrete_sequence=['#FF6B6B', '#4ECDC4', '#FFD166']
        )

        fig.update_traces(textinfo='percent+label', textposition='inside')
        fig.update_layout(title_x=0.5)

        return fig


    # ---------------------------------------------------------------------
    #  Graphique : Distribution des prix
    # ---------------------------------------------------------------------
    def create_price_distribution(self):

        fig = px.histogram(
            self.df,
            x='prix_m2',
            nbins=40,
            title="␣Distribution␣des␣prix␣au␣m  ␣en␣France",
            color_discrete_sequence=['#45B7D1'],
            opacity=0.7
        )

        avg_price = self.df['prix_m2'].mean()
        fig.add_vline(
            x=avg_price, line_dash="dash", line_color="red",
            annotation_text=f"Moyenne␣:␣{avg_price:.0f}   "
        )

        fig.update_layout(title_x=0.5)

        return fig


    # ---------------------------------------------------------------------
    #  Graphique : Top communes
    # ---------------------------------------------------------------------
    def create_top_cities_chart(self, top_n=10):

        stats = self.df["nom_commune"].value_counts().head(top_n)

        fig = px.bar(
            x=stats.values,
            y=stats.index,
            orientation='h',
            labels={'x': 'Nombre␣de␣transactions', 'y': 'Commune'},
            title=f"␣Top␣{top_n}␣communes␣par␣transactions",
            color=stats.values,
            color_continuous_scale="Viridis",
```

```
            text=stats.values
        )

        fig.update_layout(title_x=0.5)
        fig.update_traces(textposition='outside')

        return fig


    # ----------------------------------------------------------------------
    #  Graphique : Surface vs Prix
    # ----------------------------------------------------------------------
    def create_price_vs_surface(self):

        sample_df = self.df.sample(min(300, len(self.df)))

        fig = px.scatter(
            sample_df,
            x='surface_reelle_bati',
            y='prix_m2',
            color='type_local',
            title=' Relation entre surface et prix au m  ',
            labels={
                'surface_reelle_bati': 'Surface (m  )',
                'prix_m2': 'Prix au m  (   )',
                'type_local': 'Type de bien'
            },
            hover_data=['nom_commune'],
        )

        fig.update_layout(title_x=0.5)
        return fig


    # ----------------------------------------------------------------------
    #  Dashboard complet
    # ----------------------------------------------------------------------
    def create_dashboard_overview(self):

        fig = make_subplots(
            rows=2, cols=2,
            subplot_titles=[
                ' Types de biens',
                ' Distribution prix',
                ' Top communes',
                ' Surface vs Prix'
            ],
            specs=[[{"type": "domain"}, {"type": "xy"}],
                   [{"type": "xy"}, {"type": "xy"}]]
        )

        # Ajouter chaque graphique dans le dashboard
        charts = [
            self.create_property_types_chart(),
```

```
            self.create_price_distribution(),
            self.create_top_cities_chart(8),
            self.create_price_vs_surface()
        ]

        positions = [(1,1), (1,2), (2,1), (2,2)]

        for chart, pos in zip(charts, positions):
            if chart:
                for trace in chart.data:
                    fig.add_trace(trace, row=pos[0], col=pos[1])

        fig.update_layout(
            height=900,
            title="␣Dashboard␣Immobilier␣France␣-␣Vue␣d'ensemble",
            title_x=0.5
        )

        return fig



# ----------------------------------------------------------------------
#   TEST DU MODULE
# ----------------------------------------------------------------------
if __name__ == "__main__":


    # Initialiser le visualizer
    viz = Visualizer()

    # Menu de test interactif
    print("\n␣QUE␣VEUX-TU␣TESTER␣?")
    print("1.␣Camembert␣types␣de␣biens")
    print("2.␣Histogramme␣prix")
    print("3.␣Top␣communes")
    print("4.␣Graphique␣surface␣vs␣prix")
    print("5.␣Dashboard␣complet")
    print("6.␣Tout␣tester")

    try:
        choix = input("\n␣Ton␣choix␣(1-6):␣").strip()

        if choix in ['1', '6']:
            print("\n␣Test␣camembert...")
            fig1 = viz.create_property_types_chart()
            fig1.show()

        if choix in ['2', '6']:
            print("\n␣Test␣histogramme...")
            fig2 = viz.create_price_distribution()
            fig2.show()

        if choix in ['3', '6']:
```

```
            print("\n␣Test␣top␣communes...")
            fig3 = viz.create_top_cities_chart()
            fig3.show()

        if choix in ['4', '6']:
            print("\n␣Test␣surface␣vs␣prix...")
            fig4 = viz.create_price_vs_surface()
            fig4.show()

        if choix in ['5', '6']:
            print("\n␣Test␣dashboard␣complet...")
            dashboard = viz.create_dashboard_overview()
            dashboard.show()



    except Exception as e:
        print(f"␣Erreur:␣{e}")
```

5.2 Module MapVisualizer.py

```
"""
MAP␣VISUALIZER
Carte␣interactive␣des␣prix␣immobiliers␣par␣d partement␣(France)
"""

import json
import pandas as pd
import plotly.express as px
from pathlib import Path
import numpy as np # Ajout  pour les op rations de DataFrame

class MapVisualizer:

    def __init__(self):
        """Initialise␣la␣visualisation␣cartographique"""

        current_dir = Path(__file__).parent
        project_root = current_dir.parent

        data_file = project_root / "data" / "processed" / "dvf_cleaned.csv
            "
        geo_file = project_root / "data" / "geo" / "departements.geojson"

        print("Initialisation␣de␣MapVisualizer...")

        # Charger les donn es DVF
        self.df = pd.read_csv(data_file)
        self.df['prix_m2_median'] = self.df['prix_m2'].median() # Pr pare
            une colonne pour le fallback Scatter Map
        print(f"{len(self.df)}␣transactions␣charg es.")
```

```
        # Charger le GeoJSON des d partements
        with open(geo_file, "r", encoding="utf-8") as f:
            self.geojson = json.load(f)
        print("GeoJSON␣des␣d partements␣charg .")

    # ------------------------------------------------------------
    # CARTE 1: Prix moyen au m  par d partement (Choropleth, pour le
       Dashboard g n ral)
    # ------------------------------------------------------------
    def create_price_map(self):
        """
␣␣␣␣␣␣␣␣Cr e␣une␣carte␣choropl the␣du␣prix␣moyen␣au␣m  ␣par␣d partement
␣␣␣␣␣␣␣␣(pour␣la␣vue␣agr g e␣du␣Dashboard).
␣␣␣␣␣␣␣␣"""
        df_grouped = (
            self.df.groupby("code_departement")
            .agg(avg_price_m2=("prix_m2", "mean"))
            .reset_index()
        )

        fig = px.choropleth_mapbox(
            df_grouped,
            geojson=self.geojson,
            locations="code_departement",
            featureidkey="properties.code",
            color="avg_price_m2",
            color_continuous_scale="Viridis",
            mapbox_style="carto-positron",
            zoom=4.5,
            center={"lat": 46.5, "lon": 2.5},
            opacity=0.7,
            labels={"avg_price_m2": "Prix␣moyen␣(  /m  )"},
            title="Prix␣moyen␣au␣m  ␣par␣d partement␣(Vue␣agr g e)",
        )

        fig.update_layout(title_x=0.5, height=600)
        return fig

    # ------------------------------------------------------------
    # CARTE 2: Scatter Mapbox (POUR L'INTERACTIVIT  AVEC LES VILLES)
    # ------------------------------------------------------------
    def create_scatter_map(self, df_filtered: pd.DataFrame):
        """
␣␣␣␣␣␣␣␣Cr e␣une␣carte␣affichant␣les␣transactions␣individuelles
␣␣␣␣␣␣␣␣avec␣le␣nom␣de␣la␣commune␣au␣survol.

␣␣␣␣␣␣␣␣Parameters
␣␣␣␣␣␣␣␣----------
␣␣␣␣␣␣␣␣df_filtered␣:␣pd.DataFrame
␣␣␣␣␣␣␣␣␣␣␣␣DataFrame␣d  j  ␣filtr ␣par␣le␣widget␣Streamlit␣(d partement
    ,␣etc.).
␣␣␣␣␣␣␣␣"""
        # Pour des raisons de performance, on ne prend qu'un  chantillon
           de 2000 points max.
```

```
        sample_df = df_filtered.sample(min(2000, len(df_filtered)))

        # Centrer la carte sur la r gion filtr e
        if not sample_df.empty:
            center_lat = sample_df['latitude'].mean()
            center_lon = sample_df['longitude'].mean()
        else:
            center_lat = 46.5
            center_lon = 2.5

        fig = px.scatter_mapbox(
            sample_df,
            lat="latitude",
            lon="longitude",
            hover_name="nom_commune", # <--- C'EST CET    LMENT    QUI
               AFFICHE LE NOM DE LA VILLE
            hover_data=['prix_m2', 'surface_reelle_bati'],
            color="prix_m2",
            color_continuous_scale=px.colors.sequential.Plasma,
            mapbox_style="carto-positron",
            zoom=7, # Zoom initial plus serr
            center={"lat": center_lat, "lon": center_lon},
            opacity=0.8,
            title="Transactions individuelles (Nom des villes au survol)"
        )

        fig.update_layout(title_x=0.5, height=600, margin={"r":0,"t":40,"l
           ":0,"b":0})
        return fig


# ------------------------------------------------------------
# LANCE LA CARTE SI ON EX CUTE CE FICHIER DIRECTEMENT
# ------------------------------------------------------------
if __name__ == "__main__":
    viz = MapVisualizer()
    fig = viz.create_price_map()
    fig.show()
```

5.3 Application Streamlit

```
import streamlit as st
from immo_scope.visualizer import Visualizer
from immo_scope.map_visualizer import MapVisualizer
from datetime import datetime
import pandas as pd
import plotly.express as px


@st.cache_resource
def load_visualizers():
    """Instancie␣et␣met␣en␣cache␣les␣classes␣de␣visualisation."""
    try:
        # Les classes Visualizer et MapVisualizer contiennent la logique
           de chargement de dvf_cleaned.csv
        return Visualizer(), MapVisualizer()
    except Exception as e:

        return None, None

viz, map_viz = load_visualizers()

st.set_page_config(
    page_title="Immo_Scope␣Dashboard",
    page_icon="    ",
    layout="wide"
)


col_logo, col_title = st.columns([1, 6])
with col_logo:

    st.image("https://placehold.co/100x100/4ECDC4/white?text=IMMO", width
       =80)

with col_title:
    st.title("    ␣Immo_Scope␣   ␣Analyse␣du␣march ␣immobilier␣fran ais"
       )
    st.markdown("Plateforme␣interactive␣:␣**Visualisations**␣|␣**Qualit
       **␣|␣**Ing nierie␣de␣Donn es**")

st.divider() #

# ----------------------------------------------------------------------
# V RIFICATION D'ERREUR AU CHARGEMENT
# ----------------------------------------------------------------------

if viz is None or map_viz is None:
    st.error("␣Les␣modules␣de␣visualisation␣ou␣les␣donn es␣(dvf_cleaned.
       csv)␣n'ont␣pas␣pu␣ tre ␣charg s.")
    st.info("V rifiez␣l'ex cution␣du␣DataLoader␣(rapport_rodrigue.py)␣
       pour␣g n rer␣les␣donn es.")
    st.stop()
```

```
# Navigation
menu = st.sidebar.selectbox(
    "    ␣Navigation",
    ["Accueil", "Visualisations", "Carte␣interactive", "Dashboard␣complet"
        ]
)

# ----------------------------------------------------------------------
# CONTENU DYNAMIQUES
# ----------------------------------------------------------------------

if menu == "Accueil":
    st.header("1.␣Pr sentation␣et␣Bilan␣du␣Projet␣    ")

    st.markdown("""
␣␣␣␣Cette␣plateforme␣est␣le␣r sultat␣d'un␣pipeline␣complet␣de␣Data␣
    Science␣appliqu ␣aux␣donn es␣**DVF␣(Demandes␣de␣Valeurs␣Fonci res)␣
    2023**␣en␣France.
␣␣␣␣""")

    st.subheader("Objectifs␣et␣R partition␣des␣R les")

    # --- BILAN D' QUIPE
    with st.container(border=True):
        st.markdown("**Statut␣:␣Phase␣de␣Donn es␣Termin e␣&␣Dashboard␣
            Op rationnel**")
        st.markdown("""
␣␣␣␣␣␣␣␣Ce␣tableau␣r sume␣la␣contribution␣de␣chaque␣membre␣aux␣exigences␣
    cl s␣du␣projet␣:
␣␣␣␣␣␣␣␣""")

        st.table(pd.DataFrame({
            'Exigence': ['Nettoyage␣&␣POO␣Avanc e', 'Tests␣Unitaires␣(
                Qualit )', 'Dashboard␣Interactif␣(Widgets)'],
            'Responsable': ['Rodrigue', 'L  a', 'Hadjer'],
        }))

    st.subheader("R sultats␣Cl s␣de␣l'Analyse")
    try:
        df_all = viz.df
        st.info(f"**Donn es␣:**␣{len(df_all):,}␣transactions␣analys es␣
            dans␣{df_all['nom_commune'].nunique()}␣communes.")
        st.metric("Prix␣Moyen␣/␣m  ", f"{viz.data['metrics']['avg_price_m2
            ']:.0f}␣   ",
                  delta=f"Surface␣Moyenne:␣{viz.df['surface_reelle_bati'].
                      mean():.0f}␣m  ")
    except Exception:
        pass
elif menu == "Visualisations":
    st.header("2.␣Analyse␣D taill e␣des␣Tendances")

    # --- ACC S AUX DONN ES ET FILTRES (DEPUIS LE VISUALIZER) ---
    df_all = viz.df.copy()
```

```
    try:
        # Acc der au DataFrame pour calculer les villes les plus
            fr quentes
        if not df_all.empty and 'nom_commune' in df_all.columns:
            top_cities = df_all['nom_commune'].value_counts().head(5).
                index.tolist()
            st.caption(f"Analyse␣bas e␣sur␣{len(df_all['nom_commune'].
                unique())}␣communes.␣Top␣5␣:␣{',␣'.join(top_cities)}")
    except Exception as e:
        st.warning(f"Erreur␣lors␣de␣l'acc s␣aux␣donn es␣pour␣l'aper u␣:
            ␣{e}")

    st.divider()

    tab1, tab2, tab3, tab4 = st.tabs(
        ["Types␣de␣biens", "Distribution␣des␣prix", "Top␣communes", "
            Surface␣vs␣Prix"]
    )

    with tab1:
        st.subheader("R partition␣par␣type␣de␣bien")
        st.plotly_chart(viz.create_property_types_chart(),
            use_container_width=True)
        st.caption("Le␣graphique␣montre␣la␣r partition␣entre␣Maisons␣et␣
            Appartements.")

    with tab2:
        st.subheader("Distribution␣des␣Prix␣au␣m  ")
        st.plotly_chart(viz.create_price_distribution(),
            use_container_width=True)
        st.caption("Ligne␣rouge␣:␣Prix␣moyen␣national.␣Utile␣pour␣
            identifier␣les␣extr mes␣et␣la␣concentration␣des␣prix.")

    with tab3:
        st.subheader("Top␣Communes␣par␣transactions")
        st.plotly_chart(viz.create_top_cities_chart(), use_container_width
            =True)
        st.caption("Classement␣des␣villes␣ayant␣le␣plus␣grand␣volume␣de␣
            transactions␣DVF.")

    with tab4:
        st.subheader("Relation␣Surface␣(m  )␣vs␣Prix␣au␣m  ␣(   )")
        st.plotly_chart(viz.create_price_vs_surface(), use_container_width
            =True)
        st.caption("Le␣nuage␣de␣points␣permet␣de␣d tecter␣les␣
            corr lations␣ou␣les␣anomalies␣de␣prix␣par␣rapport␣  ␣la␣taille
            ␣du␣bien.")

elif menu == "Carte␣interactive":
    st.header("3.␣Carte␣Interactive␣des␣Transactions")

    # --- ACC S AUX DONN ES ET WIDGETS ---
    df_map_source = viz.df.copy()
```

```
    # 1. Widget de s lection du d partement (Pour filtrer la carte)
    try:
        departments = df_map_source['code_departement'].unique().tolist()
        departments.sort()

        selected_dept = st.selectbox(
            "Filtrer la carte par D partement :",
            options=["Tous les d partements"] + departments,
            key="map_dept_select"
        )

        # 2. Application du filtre
        if selected_dept != "Tous les d partements":
            df_filtered = df_map_source[df_map_source['code_departement']
               == selected_dept].copy()

            # Affichage des villes filtr es
            top_cities_dept = df_filtered['nom_commune'].value_counts().
               head(5).index.tolist()
            st.info(f"Les 5 villes les plus actives dans le **{
               selected_dept}** sont : **{', '.join(top_cities_dept)}**.")
        else:
            df_filtered = df_map_source

        st.markdown("---")

        # 3. Affichage de la carte

        if len(df_filtered) > 0:
            st.subheader("Transactions individuelles")

            # Appel de la m thode Scatter Map dans MapVisualizer pour les
                villes
            scatter_fig = map_viz.create_scatter_map(df_filtered)
            st.plotly_chart(scatter_fig, use_container_width=True)

            st.caption("Chaque point repr sente une transaction. Survolez
               -le pour voir le nom de la commune et les d tails.")
        else:
             st.warning("Aucune transaction trouv e pour ce filtre.")

    except Exception as e:
        st.error(f"Erreur lors de l'affichage de la carte: {e}")
        st.info("V rifiez que les classes MapVisualizer et Visualizer ont
           bien la colonne 'code_departement'.")


elif menu == "Dashboard complet":
    st.header("4. Synth se Globale du March ")

    # --- AJOUT DU FILTRE ET DE LA CARTE INTERACTIVE ---
    df_map_source = viz.df.copy()
```

```
# 1. Widget de s lection du d partement
try:
    departments = df_map_source['code_departement'].unique().tolist()
    departments.sort()

    selected_dept = st.selectbox(
        "Filtrer les transactions par D partement :",
        options=["Tous les d partements"] + departments,
        key="dashboard_dept_select"
    )

    # 2. Application du filtre
    if selected_dept != "Tous les d partements":
        df_filtered = df_map_source[df_map_source['code_departement']
           == selected_dept].copy()
        st.info(f"Filtre actif : Affichage de {len(df_filtered):,}
           transactions pour le d partement **{selected_dept}**.")
    else:
        df_filtered = df_map_source

    st.divider()

except Exception as e:
    st.error(f"Erreur lors de l'application du filtre : {e}")
    df_filtered = viz.df.copy()

# --- SYNTH SE DU PRIX MOYEN PAR D PARTEMENT (CARTE CHOROPL THE
   GLOBALE) ---
with st.container(border=True):
    st.subheader("Prix Moyen au m  par D partement (Vue Agr  g  e)")
    st.plotly_chart(map_viz.create_price_map(), use_container_width=
       True)
    st.caption("Cette carte choropl the offre une vue nationale et n'
       est pas affect e par le filtre d partemental ci-dessus.")

# --- CARTE INTERACTIVE FILTR E ---
st.subheader("Transactions Filtr es")
if len(df_filtered) > 0:
    # Appel de la m thode Scatter Map pour les villes
    scatter_fig = map_viz.create_scatter_map(df_filtered)
    st.plotly_chart(scatter_fig, use_container_width=True)
    st.caption("Carte d taill e des transactions, mise  jour par
       le filtre du d partement s lectionn .")
else:
    st.warning("Aucune transaction trouv e pour le d partement
       s lectionn .")

# --- VUE D TAILL E DU MARCH  ---
st.subheader("Rapport Multi-Dimensionnel Global")

st.plotly_chart(viz.create_dashboard_overview(), use_container_width=
   True)
st.caption("Vue compil e des principales tendances de prix et de
   r partition.")
```

```
st.sidebar.markdown("---")
st.sidebar.caption(f"Derni re mise  jour : {datetime.now().strftime('%Y
    -%m-%d %H:%M:%S')}")
```

6 Conclusion

La partie Visualisation constitue une étape essentielle du projet Immo Scope, car elle transforme les données DVF nettoyées et validées en outils concrets permettant de comprendre le marché immobilier français.

Grâce au travail réalisé, le projet dispose désormais :

— de graphiques lisibles permettant d'identifier les tendances clés du marché ;

— d'une carte interactive intuitive facilitant l'analyse géographique ;

— d'une application simple d'utilisation, structurée et optimisée.

Mon travail complète ainsi celui de Rodrigue (Data Engineering) et de Léa (Qualité & Tests), en apportant la couche finale : la **mise en valeur des données**.

L'ensemble du projet démontre comment une approche méthodique combinant POO, pipeline de données, tests unitaires et visualisation moderne peut produire un outil d'analyse professionnel et accessible.